DBMS | OAVS COMPUTER TEACHER (TGT) | STUDY NOTES

Chapter 10: Concurrency Controls

10.1 Problems of Concurrent Transactions; Control Mechanisms such as Locks, Time-Stamps, Optimistic Scheduling and MVT

▸ What concurrency is and why control is needed | ▸ Lost Update, Dirty Read, Unrepeatable Read, Incorrect Summary
▸ Lock-Based Protocols (S/X locks, compatibility, 2PL) | ▸ Timestamp Ordering Protocol
▸ Optimistic (Validation-Based) Concurrency Control | ▸ Multiversion Timestamp Ordering (MVT)

Contents

Intro: Concurrent Transactions & Need for Control

1. Problems of Concurrent Transactions (Lost Update, Dirty Read, Unrepeatable Read, Incorrect Summary)
2. Lock-Based Concurrency Control (S / X Locks, Compatibility, Two-Phase Locking)
3. Time-Stamp Based Concurrency Control
4. Optimistic Scheduling (Validation-Based Protocol)
5. Multiversion Timestamp Ordering (MVT)
6. Quick Revision / Cheat Sheet

Concurrent Transactions & Need for Control

What is Concurrency Control?

Concurrency Control is the set of techniques (protocols) used by a DBMS to manage the simultaneous execution of transactions on a shared database, so that the database always remains **consistent** and every transaction stays **isolated** from the others.

- It controls the **interleaving** of read/write operations of concurrent transactions.
- Its aim is **Serializability**: the final result of the interleaved schedule must be the same as the result of some serial (one-after-another) execution of the same transactions.
- The **Concurrency Control Manager** enforces it using Locks, Time-Stamps, Optimistic Scheduling and MVT.

INTERLEAVING
Interleaving means the operations of two or more transactions are executed alternately during the same time period, instead of one transaction finishing completely before the next begins.

Why is Concurrency Control Required?

- To prevent Lost Update, Dirty Read, Unrepeatable Read and Incorrect Summary.
- To preserve **consistency** (integrity constraints hold before and after execution).
- To guarantee **Isolation** (the I of ACID) — no transaction sees another's partial results.
- To allow high concurrency (performance) *without* sacrificing correctness.

REMEMBER
Concurrency Control = DBMS protocols that keep concurrent transactions equivalent to a serial execution. It balances **maximum concurrency** (performance) against **data consistency** (correctness) — Locks, Timestamps, Optimistic and MVT are four ways to strike this balance.

1. Problems of Concurrent Transactions

When two or more transactions access shared data concurrently *without* proper control, four classic problems can occur. In all schedules below, `R(X)` = read item X, `W(X)` = write item X, T1 and T2 are two transactions, and time flows downward.

1.1 Lost Update Problem

Occurs when two transactions read the same data item and then update it based on the value they read; the update made by the first transaction gets **overwritten (lost)** by the second.

Schedule 1 — Lost Update

Time	T1	T2	Value of X	Remark
t1	`R(X) → 100`		100	
t2		`R(X) → 100`	100	**Both read the same old value**
t3	`X = X − 50 = 50`		100	
t4	`W(X)`		50	**T1 writes 50**
t5		`X = X + 20 = 120`	50	
t6		`W(X)`	120	**✗ T1's update LOST**

Here, T1's write of X=50 is completely lost because T2 had already read the old value (100) before T1's write, and T2 overwrites X with 120. The withdrawal done by T1 disappears from the database. Any serial execution would give **X = 70** (100 − 50 + 20), so the final value 120 is wrong.

> **COMMON MISTAKE**
> Students often think "Lost Update" means data is deleted. It actually means one transaction's **write operation is overwritten** by another transaction's write, based on a stale read — the update is logically lost, not physically deleted.

1.2 Dirty Read / Temporary Update Problem

Occurs when a transaction reads a value written by another transaction that has **not yet committed** (and later that other transaction fails/rolls back). The value read is "dirty" — it never really existed in the committed database.

Schedule 2 — Dirty Read

Time	T1	T2	Value of X	Remark
t0	`R(X) → 100`		100	**Initial value: X = 100**
t1	`W(X) → 50`		50	**Not yet committed**
t2		`R(X) → 50`	50	**✗ T2 reads UNCOMMITTED value**
t3	`ROLLBACK`		100	**X restored to 100**
t4		`uses X = 50`	100	**WRONG (dirty) value used**

T2 has read a value that was never committed. Once T1 rolls back, the database reverts to X = 100, but T2 has already used X = 50 in its calculations — this is called a **dirty read** (also called reading a **temporary update**, since T1's update was only temporary).

> **EXAM POINT**
> Dirty Read is possible only when a transaction is allowed to read **uncommitted** data of another active transaction. Preventing dirty reads is directly related to the **Isolation** property of ACID.

1.3 Unrepeatable Read Problem (Inconsistent Retrieval)

Occurs when a transaction reads the **same data item twice** and gets two **different values**, because another transaction modified and committed the item in between the two reads.

Schedule 3 — Unrepeatable Read

Time	T1	T2	Value of X	Remark
t1	R(X) → 100		100	First read
t2		W(X) → 200	200	
t3		COMMIT	200	Change is committed
t4	R(X) → 200		200	✗ Different value than t1

T1 read X twice within the same transaction and got two different results (100, then 200). This violates the expectation that a transaction should see a consistent view of data throughout its execution.

> **REMEMBER**
> Unrepeatable Read ≠ Dirty Read. In Dirty Read, the value read was never committed. In Unrepeatable Read, both values are committed values, just read at different times.

1.4 Incorrect Summary / Inconsistent Analysis Problem

Occurs when one transaction is computing an **aggregate summary function** (e.g. SUM, COUNT) over a set of data items while another transaction is **updating some of those items** in between — the aggregate transaction reads some values before the update and some after, giving a wrong total.

Schedule 4 — Incorrect Summary (T1 computes SUM of A, B, C; T2 transfers 10 from C to A)

Time	T1 (SUM)	T2 (Transfer)	A	B	C	Remark
t0			50	30	20	True sum = 100
t1	R(A) → 50 Sum = 50		50	30	20	
t2		R(C); C = C − 10 W(C)	50	30	10	
t3		R(A); A = A + 10 W(A); COMMIT	60	30	10	Transfer done, total still 100
t4	R(B) → 30 Sum = 80		60	30	10	
t5	R(C) → 10 Sum = 90		60	30	10	✗ Sum = 90, should be 100

The true sum is **100** (50 + 30 + 20), and T2's transfer of 10 from C to A does not change it. But T1 read A *before* T2's update (old A = 50) and C *after* T2's update (new C = 10), so it gets 50 + 30 + 10 = **90**. This value matches no consistent state of the database. That is why the problem is called **Incorrect Summary** or **Inconsistent Analysis**.

> **SHORTCUT TO REMEMBER THE 4 PROBLEMS**
> **L-D-U-I:** **L**ost Update (overwritten write) → **D**irty Read (reads uncommitted data) → **U**nrepeatable Read (same read, two different committed values) → **I**ncorrect Summary (aggregate reads a mixed/inconsistent state).

Table 1.1 — Comparison of the Four Problems

Problem	Cause	Key Symptom
Lost Update	Two transactions write based on the same stale read	One transaction's write is overwritten / disappears
Dirty Read	Reading data written by an uncommitted transaction	Value used never existed in committed DB
Unrepeatable Read	Re-reading same item after another transaction commits a change	Same read returns two different committed values
Incorrect Summary	Aggregate function computed while data is being updated concurrently	Aggregate result does not reflect any single consistent state

2. Lock-Based Concurrency Control

A **lock** is a variable associated with a data item that controls what operations can be performed on that item by concurrent transactions. Before accessing a data item, a transaction must request the appropriate lock from the **Lock Manager**. If the lock is already held by another transaction in an incompatible mode, the requesting transaction must **wait**.

2.1 Types of Locks

Lock Type	Also Called	Meaning
Shared Lock (S)	Read Lock	Allows the transaction to only **read** the data item. Multiple transactions may hold a shared lock on the same item simultaneously.
Exclusive Lock (X)	Write Lock	Allows the transaction to both **read and write** the data item. Only one transaction may hold an exclusive lock on an item, and no other lock (S or X) can be held at the same time by anyone else.

Basic Locking Rule

T wants to Read X → Request **Lock-S(X)** → Granted if no one holds X(X)
T wants to Write X → Request **Lock-X(X)** → Granted if no one holds S(X) or X(X)

Fig 2.1 — Requesting locks before read/write operations

2.2 Lock Compatibility Matrix

A lock request for item X by transaction Ti is granted only if it is **compatible** with the locks already held on X by other transactions.

Table 2.1 — Lock Compatibility Matrix

	S (Shared)	X (Exclusive)
S (Shared)	✓ **Compatible**	✗ **Not Compatible**
X (Exclusive)	✗ **Not Compatible**	✗ **Not Compatible**

EXAM POINT
Only **S–S is compatible**. Any combination involving X is incompatible. This single row/column fact is a very common one-mark MCQ.

2.3 How Locks Control Concurrent Access

- Before any Read(X), a transaction obtains `Lock-S(X)`.
- Before any Write(X), a transaction obtains `Lock-X(X)` (or upgrades an existing S lock to X).
- After finishing use of X, the transaction executes `Unlock(X)` to release it.
- If a requested lock is not compatible with an existing lock, the requesting transaction is made to **wait** in a queue until the lock is released.
- This mutual exclusion prevents Lost Update and Dirty Read directly, because a writer gets exclusive access and readers cannot see uncommitted writes while the X lock is held.

Two-Phase Locking (2PL) — brief control rule

Simply acquiring/releasing locks anywhere is not enough to guarantee serializability. The **Two-Phase Locking Protocol** requires every transaction to have exactly two phases:

Growing Phase — locks may only be *acquired*, none released

↓

Lock Point — transaction holds its maximum number of locks

↓

Shrinking Phase — locks may only be *released*, none acquired

Fig 2.2 — Two-Phase Locking: Growing then Shrinking phase

REMEMBER

A schedule in which every transaction follows 2PL is guaranteed to be **conflict-serializable** (in short: equivalent to some serial schedule). Once a transaction releases even a single lock, it can never acquire a new lock afterwards.

COMMON MISTAKE

2PL does *not* mean "lock twice." It means locking happens in **two phases** — a growing phase and a shrinking phase — for the whole transaction.

SHORTCUT

Lock type memory trick: **S = Share = many Same-time readers OK**; **X = eXclusive = eXactly one** transaction, no one else at all.

3. Time-Stamp Based Concurrency Control

In this method, every transaction Ti is assigned a unique **timestamp TS(Ti)** at the moment it enters the system — usually the system clock value or a logical counter. Older transactions get smaller timestamps than younger ones. Timestamps are used to decide the **order** in which conflicting operations should take place, *without using any locks at all*.

REMEMBER

Basic Rule: Conflicting operations of two transactions are executed in **timestamp order** — i.e., the effect must be the same as if the transaction with the smaller (older) timestamp had executed first.

EXAMPLE

Suppose T1 enters the system at 10:00:05 and T2 enters at 10:00:08. The system assigns **TS(T1) = 10** and **TS(T2) = 20** (larger counter value = later entry). Since TS(T1) < TS(T2), T1 is treated as the **older** transaction and T2 as the **younger** one. If T1 and T2 both want to access the same data item X, the protocol makes sure the result looks as if T1's operations happened before T2's — exactly their timestamp order.

3.1 Timestamps Associated with Each Data Item

For every data item X, the system maintains two timestamp values:

Notation	Meaning
W-timestamp(X)	The largest timestamp of any transaction that successfully executed Write(X)
R-timestamp(X)	The largest timestamp of any transaction that successfully executed Read(X)

3.2 Basic Timestamp Ordering (TO) Rules

When Ti issues Read(X):

- If TS(Ti) < W-timestamp(X) → Ti is trying to read a value that was already overwritten by a "younger" transaction → **Reject** and roll back Ti.
- Otherwise → execute Read(X) and set R-timestamp(X) = max(R-timestamp(X), TS(Ti)).

When Ti issues Write(X):

- If TS(Ti) < R-timestamp(X) → a younger transaction already read X and expects the old value → **Reject** and roll back Ti.
- If TS(Ti) < W-timestamp(X) → a younger transaction already wrote a newer value → **Reject** this obsolete write (Thomas Write Rule may simply ignore it instead of rolling back).
- Otherwise → execute Write(X) and set W-timestamp(X) = TS(Ti).

WORKED EXAMPLE

Let T1 be older with **TS(T1) = 10**, and T2 be younger with **TS(T2) = 20**. Item X starts with R-timestamp(X) = 0 and W-timestamp(X) = 0.

Step	Operation	Rule Check	R-ts(X)	W-ts(X)	Result
1	T1: Read(X)	TS(T1)=10 ≥ W-ts(X)=0	10	0	**Allowed**
2	T2: Write(X)	TS(T2)=20 ≥ R-ts(X)=10 and ≥ W-ts(X)=0	10	20	**Allowed**
3	T1: Write(X)	TS(T1)=10 < W-ts(X)=20	10	20	✗ **Rejected**

At Step 3, T1 tries to write X, but a younger transaction (T2) has already written a newer version. Allowing T1's write now would silently erase T2's newer value, so T1 is **rejected and rolled back** — it must restart with a fresh, larger timestamp.

EXAM POINT

Timestamp Ordering guarantees serializability equivalent to the **serial order of the timestamps themselves** — no waiting is needed, but transactions may be **restarted (rolled back)** more often, and a rolled-back transaction is given a **new, larger timestamp** when restarted (to avoid starvation).

IMPORTANT NOTE

Timestamp ordering does **not** use locks and does **not** cause deadlock — because a transaction is never made to wait; it is either allowed to proceed or immediately rolled back.

SHORTCUT

"**An older transaction must not be allowed to act as if a younger transaction had not already read or written the same item.**" If Ti is too old to make sense of the current state (something younger already happened), reject Ti.

4. Optimistic Scheduling (Validation-Based Protocol)

Optimistic Concurrency Control **assumes conflicts are rare**. Instead of checking for conflicts on every read/write (as locks or timestamps do), a transaction is allowed to execute **freely on its own local copy of data** and is checked for conflicts only at the **end**, just before it commits. If no conflict is found, changes are made permanent; otherwise the transaction is **rolled back and restarted**.

REMEMBER

Optimistic protocols work best when the database is mostly used for **reads**, so that the probability of two transactions actually conflicting is low, making rollbacks rare and expensive locking unnecessary.

4.1 The Three Phases

1. Read Phase

Transaction reads values of data items from DB into local (private) variables and performs all computations locally. No updates are made to the actual database yet.

↓

2. Validation Phase

Before writing, the system checks whether this transaction's read/write set conflicts with other transactions that were concurrently active. Validation tests are applied using timestamps assigned at the start of each phase.

↓

3. Write Phase

If validation succeeds, all local updates are applied (made permanent) to the actual database. If validation fails, the transaction is aborted and restarted (its local changes are simply discarded).

Fig 4.1 — Read → Validation → Write phases of Optimistic Concurrency Control

4.2 Simple Working Example

```
T1: Read Phase  -> reads X locally, computes X' = X + 10
T2: Read Phase  -> reads X locally, computes X' = X - 5      (runs concurrently with T1)

T1: Validation  -> checks: did any concurrent transaction already WRITE X?  No -> PASS
T1: Write Phase -> X is updated in the database

T2: Validation  -> checks: did any concurrent transaction (T1) already WRITE X after T2 read it? YES
T2: Write Phase -> NOT executed. T2 is ROLLED BACK and must restart with fresh read of X.
```

EXAM POINT

Optimistic control has **no locking overhead** and **no deadlock** (since resources are never locked while waiting), but it can lead to **wasted work** and repeated restarts if the actual conflict rate is higher than expected — so it suits low-conflict (mostly read-only) workloads.

SHORTCUT

Remember the 3 phases by the acronym **R-V-W**: **R**ead locally → **V**alidate at the end → **W**rite only if validation passes.

5. Multiversion Timestamp Ordering (MVT)

Instead of overwriting a data item on every Write, the **Multiversion Timestamp (MVT)** scheme keeps **multiple versions** of each data item — one version per successful write. Every version carries the timestamp of the transaction that created it. This lets Read operations be directed to the correct **older version** instead of being rejected, greatly reducing unnecessary rollbacks compared to basic timestamp ordering.

REMEMBER

Core Idea: **"Never say no to a Read."** Because old versions are kept, a Read can almost always be satisfied by picking the right version — the newest version whose timestamp is still ≤ the reader's own timestamp.

5.1 Structure of Each Version

Each version of data item X, written as X_i, stores three fields:

Field	Meaning
Content(X_i)	The actual value/content of this version
W-timestamp(X_i)	Timestamp of the transaction that created (wrote) this version
R-timestamp(X_i)	Largest timestamp among all transactions that have successfully read this version

5.2 Basic Working Rules

When Ti issues Read(X): find the version X_k having the largest W-timestamp(X_k) that is still ≤ TS(Ti); return Content(X_k) and set R-timestamp(X_k) = max(R-timestamp(X_k), TS(Ti)). — **A Read is (almost) never rejected.**

When Ti issues Write(X): let X_k be the version with the largest W-timestamp ≤ TS(Ti).

- If R-timestamp(X_k) > TS(Ti) → some younger transaction has already read a value that Ti's write would have changed → **Reject** Ti (roll back).
- Otherwise → **create a new version** X_j with Content = Ti's new value and W-timestamp(X_j) = TS(Ti); the old versions are left untouched for other readers.

WORKED EXAMPLE

Start with version X_0: Content = 100, W-ts(X_0) = 0, R-ts(X_0) = 0.

Step	Operation	Version Used	Check	Result
1. T1 (TS=10)	Write(X) = 150	Creates X_1, W-ts=10	No younger read of X_0 yet (R-ts=0 ≤ 10)	**Allowed**
2. T2 (TS=15)	Read(X)	Reads X_1 (largest W-ts ≤ 15)	Returns 150; sets R-ts(X_1) = max(0,15) = 15	**Allowed**
3. T3 (TS=12)	Write(X)	Would use X_1 (largest W-ts ≤ 12)	R-ts(X_1)=15 > TS(T3)=12	✗ **Rejected**

T3 is rejected at Step 3 because T2 (a *younger* transaction, TS=15) has already read version X_1. If T3 were allowed to overwrite X_1 now, it would silently invalidate the value T2 already read, so T3 is **rolled back** instead. Notice that T2's Read at Step 2 was never in danger of rejection — only the Write at Step 3 is affected.

Versions of X over time (each write creates a NEW version, old ones are kept)

X_0 (W-ts=0, initial) → X_1 (W-ts=10, by T1) → X_2 (W-ts=25, by T4)

A transaction with TS = 15 reading X will be given **X_1** (the newest version with W-ts ≤ 15), even though X_2 exists.

Fig 5.1 — Multiple versions of item X; a reader is served the correct "as-of" version

EXAM POINT

MVT trades **extra storage** (for keeping old versions) for a **drastic reduction in Read rejections** compared with plain Timestamp Ordering. Writes can still be rejected, but reads almost never are.

COMMON MISTAKE

MVT does not keep versions forever in practice — old versions that no active transaction can possibly need any more are eventually removed (garbage collected). Students should not assume infinite storage is used blindly; the exam-relevant point is only that *multiple* versions exist at a time, not that all history is kept eternally.

SHORTCUT

MVT = "Many Versions, To avoid rejecting Reads." Reads pick an old version; only Writes get rejected/rolled back.

Table 5.1 — Basic Timestamp Ordering vs. MVT

Aspect	Basic Timestamp Ordering	MVT
No. of versions kept	Only one (current) value	Multiple versions per item
Read rejected?	Yes, if too old (TS < W-ts)	Almost never — correct old version is used
Write rejected?	Yes, if TS < R-ts or TS < W-ts	Yes, if a younger transaction already read the relevant version
Storage overhead	Low	Higher (extra versions stored)

Quick Revision / Cheat Sheet

Chapter 10 — Concurrency Controls | One-page recap for last-minute revision

Key Definitions

- **Concurrency Control:** mechanism ensuring correctness (serializability) when transactions run in parallel.
- **Serializability:** concurrent schedule's effect = some serial schedule's effect.
- **Lock:** variable controlling access to a data item (S or X).
- **Timestamp:** unique value assigned to a transaction marking its "age"/order.
- **Version:** a snapshot of a data item's value tagged with a write-timestamp (used in MVT).

4 Problems (L-D-U-I)

- **Lost Update:** a write is overwritten due to stale reads by two transactions.
- **Dirty Read:** reading data written by an uncommitted transaction.
- **Unrepeatable Read:** same item read twice gives two different committed values.
- **Incorrect Summary:** aggregate computed over a mix of old & new values.

Locks — Rules

- S (Shared) = Read only, many readers allowed together.
- X (Exclusive) = Read+Write, only one transaction at a time.
- Compatible only: S–S. All others (S–X, X–S, X–X) incompatible.
- 2PL: Growing phase (acquire only) then Shrinking phase (release only) → guarantees serializability.

Timestamp Ordering — Rules

- Each item keeps W-timestamp(X) and R-timestamp(X).
- Read rejected if TS(Ti) < W-ts(X).
- Write rejected if TS(Ti) < R-ts(X) or TS(Ti) < W-ts(X).
- No waiting, no deadlock — rejected transactions restart with a new (larger) timestamp.

Optimistic Scheduling — Phases

- **R**ead phase: work on local copies only.
- **V**alidation phase: check for conflicts before commit.
- **W**rite phase: commit changes only if validation passes; else rollback & restart.
- Best when conflicts are rare (read-heavy workloads); no locking overhead.

MVT — Rules

- Every Write creates a **new version** instead of overwriting.
- Read is given the newest version with W-ts ≤ TS(Ti) — almost never rejected.
- Write rejected only if a younger transaction already read the relevant version.
- Trades extra storage for far fewer read rejections vs. plain timestamp ordering.

10 One-Line Exam Facts

1. Concurrency control ensures interleaved execution is equivalent to some serial schedule (serializability).
2. Lost Update happens when two transactions write based on the same stale (old) read.
3. Dirty Read = reading data from an uncommitted transaction that may later roll back.
4. Unrepeatable Read = same read within a transaction returns two different committed values.
5. Incorrect Summary = aggregate function reads a mix of pre- and post-update values.
6. Only S–S lock combination is compatible; every other pair (involving X) is incompatible.
7. Two-Phase Locking = Growing phase (only acquire) + Shrinking phase (only release).
8. In Timestamp Ordering, older-vs-younger conflicts cause the offending transaction to be rejected/rolled back — never made to wait.
9. Optimistic control follows Read → Validate → Write; best suited to low-conflict, read-heavy systems.
10. MVT keeps multiple versions per item so that Reads are (almost) never rejected — only conflicting Writes are.